

BAB homepage code

What lives in the homepage, how it loads, and how each section is used. File:
frontend/app/[locale]/page.tsx

1. What this file is

This is the public home route for English (/) and Arabic (/ar). It does two jobs:

- 1. **SEO metadata** — title and description for Google and Bing.
- 2. **Page body** — a vertical stack of homepage sections.

The site header and footer are **not** in this file. They come from the locale layout that wraps every public page: frontend/app/[locale]/layout.tsx .

2. What loads first

- 1. locale is en or ar .
- 2. The page loads homepage copy from the CMS (fetchHomepageContent) and site settings (SEO titles, FAQs).
- 3. generateMetadata chooses the search title and description in this order: homepage CMS fields, then Admin SEO settings, then a hard-coded fallback.
- 4. If FAQs exist, the page injects FAQ JSON-LD so search engines can show Q&A rich results.

Words and images are not hard-coded in page.tsx . They come from homepage CMS data (types in frontend/lib/homepage-content.ts and the admin home form). English vs Arabic is chosen by locale.

3. Sections on the page (top to bottom)

Order	Component	What visitors see
1	HeroSection	Headline, intro, call-to-action, sliding images
2	OurWorksSection	“What we do” cards (productivity, experience, reporting)
3	ExperienceSection	Connect / engage / analyze pillars
4	IndustriesSection	Food, government, healthcare, insurance, retail, plus stats
5	TransformationSection	Digital transformation block and image

Order	Component	What visitors see
6	ChannelsSection	Omnichannel headline and body
7	PartnersSection	Partner logos and copy
8	TestimonialsSection	Customer quotes
9	FaqSection	Homepage FAQ (CMS, or site settings if CMS FAQ is empty)
10	ImpactSection	Impact / results call-to-action
11	CtaContactSection	Final "get in touch" strip

4. Main page code

```
export default async function Page({ params }) {
  const { locale } = await params
  setRequestLocale(locale)
  const loc = locale === 'ar' ? 'ar' : 'en'
  const [messages, home] = await Promise.all([
    getMessages(),
    getHomepageContent(loc),
  ])
  const faqs = home.faq.items.length
    ? home.faq.items
    : faqsForLocale(siteSettings, loc)

  return (
    <>
      {faqId ? <script type="application/ld+json" /> : null}
      <HeroSection content={home.hero} />
      <OurWorksSection title={home.works.title} />
      <ExperienceSection works={home.works} experience={home.experience} />
      <IndustriesSection industries={home.industries} stats={home.stats} />
      <TransformationSection content={home.transformation} />
      <ChannelsSection content={home.channels} />
      <PartnersSection content={home.partners} />
      <TestimonialsSection title={home.testimonials.title} />
      <FaqSection faqs={faqs} />
      <ImpactSection content={home.impact} />
      <CtaContactSection content={home.cta} />
    </>
  )
}
```

5. HomeRichText helper

File: `frontend/components/home-rich-text.tsx`. This is **not** the whole homepage. It is a small parser used on Channels and Testimonials titles.

Admins can type markup in CMS text:

- `<accent>this text</accent>` — primary (brand) color
- `<muted>this text</muted>` — navy
- `<br>` — line break

The helper turns that string into React nodes so headings can have colored words without a full HTML editor.

6. Related files

File	Role
<code>frontend/app/[locale]/page.tsx</code>	Homepage route (this document)
<code>frontend/app/[locale]/layout.tsx</code>	Header, footer, fonts, analytics wrapper
<code>frontend/lib/homepage-cms.ts</code>	Fetches homepage content from the API
<code>frontend/lib/homepage-content.ts</code>	Types and default structure of homepage fields
<code>frontend/components/home-rich-text.tsx</code>	Accent / muted / line-break markup
<code>frontend/components/*-section.tsx</code>	Each visual block listed above